Chapter 3: Methodology

This chapter describes how the Hybrid Baybayin Character and Stroke Validation System will be developed and evaluated. The system checks whether a handwritten Baybayin character was formed according to the writing conventions in Pinagyamang Baybayin [2]. The chapter covers the research design, locale, participants and data sources, instruments, data gathering, preprocessing, feature extraction, model implementation, performance evaluation, the system development method, and ethical considerations. Each step states its input, the activities performed, and its output, so that other researchers can repeat the study.

3.1 Research Design

The study will use a Research and Development (R&D) design with an experimental performance evaluation. R&D is used because the study creates a new product, a validator for handwritten Baybayin characters. The evaluation is experimental because the validators are tested under the same conditions: the same data, the same split by writer, and the same metrics. Any difference in results can then be credited to the validator and not to the data.

The system is built in two connected versions:

- Version 1, the form validator, checks whether the finished drawing resembles the intended character. It uses only the final image, as the recognition studies in Chapter 2 do [7, 8, 13], and it serves as the baseline.
- Version 2, the stroke validator, checks stroke count, order, direction, trajectory, and kudlit placement. It is the improved model of the study.

A knowledge graph and a retrieval layer supply the KWF rule and source that explain each result. They do not decide the result. The decision is made only by the measurable checks of Version 1 and Version 2.

Figure 2 shows the four phases of the study.

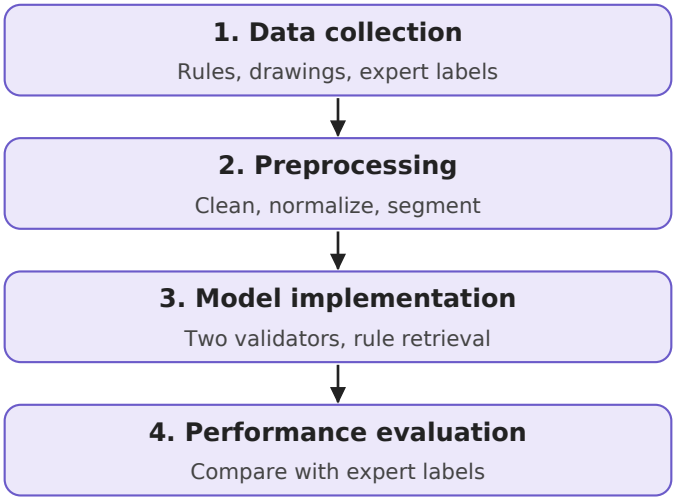


Figure 2: Research Process of the Study

The first phase, data collection, gathers the KWF reference material, the handwritten samples of volunteer writers, and the labels of the expert reviewer. It produces the character specification, the knowledge base, and a labeled dataset (Objective 1).

In the second phase, preprocessing, the raw drawings and event records are cleaned, normalized, segmented into strokes, and resampled so that samples can be compared fairly (Objective 1).

In the third phase, model implementation, features are extracted, and Version 1, Version 2, and the evidence retrieval are built (Objective 2).

In the fourth phase, performance evaluation, each validator is tested on writers held out from development and compared with the expert-reviewed labels (Objective 3).

3.2 Locale of the Study

The study will be carried out at Mariano Marcos State University (MMSU), College of Computing and Information Sciences (CCIS), City of Batac, Ilocos Norte. The researchers develop the system here, and the writing sessions will be held here using the same type of touchscreen device.

3.3 Participants and Data Sources

Three groups of sources will be used (Table 9).

Table 9: Participants and Data Sources

Source	Description	Role in the study	Number
Volunteer writers	Adults (18 years or older) with no, basic, or experienced knowledge of Baybayin who can write on a touchscreen	Write the target characters correctly and with controlled errors	20 (target)
Expert reviewer	A qualified Baybayin orthography expert	Approves the character specification and rubric, confirms the reference strokes, and labels every sample	1 (a second reviewer if available)
KWF documentary sources	Pinagyamang Baybayin (Version 3) [2], the writing guide provided by the expert [30], and the symbol files with the correct strokes	Reference standard for the character specification and the knowledge graph	Not applicable

Writers are invited by the researchers, and each signs an informed consent form. Their Baybayin experience level is recorded because it may affect the kinds of mistakes they make.

3.4 Research Instruments

Table 10: Research Instruments

Instrument	Description	Used for
Drawing canvas	A web page with a 400 × 400 pixel canvas and a fixed pen width of 8 pixels. It saves the final image and the stroke records.	Tracing references, capturing samples, and running the prototype
Task sheet	A randomized list of target characters with the instruction for each attempt: write correctly or make a stated error	Guiding each writing session
Expert checklist	A form for each sample with five yes/no checks (form, count, order, direction, kudlit placement), an overall valid or invalid label, and remarks	Ground-truth labeling
Knowledge graph	A database of character records, reference images, writing rules, kudlit rules, common errors, and sources	Evidence retrieval
Evaluation scripts	Python scripts for the metrics, confusion matrices, and timing	Performance evaluation
Consent form	A written consent form with an information sheet	Ethical compliance

Each attempt is saved as one record with these fields: sample ID, writer code, experience level, device and input type (finger or stylus), character ID, attempt number, the instruction given, the strokes, and the path of the final image. A stroke is an ordered list of points, each with x, y, and time in milliseconds. A pen-down event marks where a stroke starts and a pen-up event marks where it ends.

3.5 Data Gathering Procedure

3.5.1 Character Specification and Expert Rubric

Input. Pinagyamang Baybayin [2], the expert's writing guide [30], and the symbol files.

Activities.

1. With the expert, choose the initial character set. The default is the titik-batayan characters plus the kudlit forms of a few selected consonants. The final list is placed in Appendix A.
2. Write a specification for each character: character ID, name, KWF category, number of strokes, the order and direction of each stroke, which stroke is the kudlit (if any), and the source page.
3. A researcher traces every reference stroke on the drawing canvas by following the guide, in the stated order and direction. The expert checks each trace against the guide and approves it. If the guide accepts more than one way of writing a character, each way is stored as a separate reference.
4. Mark characters whose strokes have nearly identical shapes. Order cannot be told from shape for these characters, so order is not scored for them.
5. Write the validation rubric. A sample is valid when none of the five error categories in Table 11 applies, and invalid otherwise.

Output. Approved character specification, reference strokes, and rubric.

Table 11: Error Categories

Category	What is wrong	Instruction to the writer for a controlled error	Expected to be seen by Version 1?
Form	The shape of a stroke or of the whole character differs from the reference	Change the shape of one stroke, for example curve a straight stroke	Yes
Count	A stroke is missing or added	Leave out one stroke, or add one extra stroke (alternating by writer number)	Partly
Order	Strokes are drawn in a different sequence	Draw the strokes in a different order	No, the final image is the same
Direction	A stroke is drawn from the wrong end	Draw one stroke in the opposite direction	No, the final image is the same
Kudlit placement	The kudlit is on the wrong side or too far from the main character	Put the kudlit on the opposite side	Partly

3.5.2 Knowledge Base Construction

Input. The approved character specification and reference images.

Activities. Knowledge graphs use graph-based data models together with languages for querying and validating the stored knowledge [31]. The knowledge graph of this study stores the records in Table 12. Every record is linked to at least one Source node, so every explanation can be traced to the KWF material. Only rules, reference images, common errors, and sources are stored. Tutorials and instructional media are outside the scope of the study. The graph is stored as node and edge tables in the project database and is queried by the back

end.

Table 12: Structure of the Knowledge Graph

Node type	Attributes	Linked to
Character	ID, name, KWF category	Stroke, KudlitRule, ReferenceImage, ErrorType, Source
Stroke	Order number, direction description, reference trajectory	Character
KudlitRule	Type (above, below, cross), resulting sound, expected relative position	Character
ReferenceImage	File path, description	Character
ErrorType	Name (form, count, order, direction, kudlit), description, feedback template	Character
Source	Document, edition, page	All other node types

Output. A queryable, source-traceable knowledge base.

3.5.3 Writing Task and Sample Collection

Input. The approved specification, task sheets, volunteer writers, and the drawing canvas.

Activities.

1. Each writer signs the consent form and states their Baybayin experience (none, basic, or experienced).
2. The researcher explains the task, and the writer practices on a blank canvas.
3. For each target character, the writer sees the reference image with numbered strokes and arrows for order and direction. The writer then makes four attempts: two correct attempts and two controlled-error attempts.
4. The two error categories are assigned in rotation so that each category appears equally often for each character. Let A_c be the list of error categories that apply to character c (from its specification), w the writer number, and c the character number, both starting at 0. The two controlled errors are $A_c[(w + c) \bmod |A_c|]$ and $A_c[(w + c + 1) \bmod |A_c|]$.
5. The order of the characters is randomized for each writer using a fixed random seed that is recorded.
6. Each attempt is saved as one record. If a writer makes an accidental mark or a technical problem occurs, the attempt is repeated and the discarded attempt is logged.

Output. Raw sample records. The number of samples is $W \times C \times 4$, where W is the number of writers and C the number of characters. For example, 20 writers and 24 characters give 1,920 samples before kudlit forms are added.

3.5.4 Expert Labeling

Input. The raw sample records.

Activities.

1. The labeling page shows, for each sample, the final image and a replay of the strokes with numbers, start dots, and arrows. It hides the writer code and the instruction given, so the expert does not know whether an attempt was meant to be correct or wrong.
2. The expert completes the checklist in Table 10: five yes/no checks and an overall valid or invalid label.
3. The expert's label is the final label (the ground truth). The agreement between the instruction given and the expert's label is reported. Attempts that were meant to be correct but were labeled invalid are natural errors and are reported separately.
4. As a consistency check, 10% of the samples, chosen at random, are labeled again by the same expert at least 7 days later, and the percent agreement is reported.

Output. An expert-reviewed stroke-level dataset.

3.6 Data Cleaning and Preprocessing

Raw handwriting differs in size, position, and writing speed. None of these decide whether a character is correct, so they are removed before the validators are used.

3.6.1 Removing Incomplete Records

Input. Raw sample records.

Activities. Remove a record if a field is missing, if it has no strokes, or if the time values of its points do not increase. Very short strokes are kept, because a kudlit can be a dot. The expert marks samples with accidental marks during labeling, and those attempts are repeated.

Output. Complete records.

3.6.2 Image Normalization (Version 1)

Input. The saved final image of each complete record.

Activities. Crop the image to the bounding box of the ink, scale it so that the longest side is 56 pixels while keeping the aspect ratio, place it in the center of a 64 × 64 image, and turn it into a binary image (ink = 1, background = 0). The reference images are drawn from the reference strokes with the same pen width and are processed in the same way.

Output. Normalized binary images.

3.6.3 Stroke Segmentation, Normalization, and Resampling (Version 2)

Input. Strokes of the complete records.

Activities. A stroke is the trace between a pen-down and the next pen-up [18]. Each stroke is smoothed with a 3-point moving average. The whole character is then moved and scaled into a unit box, keeping its aspect ratio, so that the position of the kudlit relative to the main body is preserved. Each stroke is resampled to N = 32 points at equal arc-length intervals, which

removes differences in writing speed, the same idea as in [17]. Reference strokes go through the same steps. Algorithm 1 shows the procedure.

```
Algorithm 1: Preprocess
Input : raw strokes S, number of points N = 32, smoothing window k = 3
Output: normalized strokes S'

1:  for each stroke s in S do
2:      s ← moving_average(s, k)              // first and last points kept
3:  end for
4:  (x_min, y_min, x_max, y_max) ← bounding box of all points in S
5:  size ← max(x_max − x_min, y_max − y_min); if size = 0 then size ← 1
6:  for each stroke s in S do
7:      s ← ((x − x_min) / size, (y − y_min) / size) for every point
8:      s ← resample(s, N)                    // equal arc-length steps
9:  end for
10: return S
```

Big O. Let n be the total number of raw points. Each pass visits every point a fixed number of times, and resampling costs O(n + mN) for m strokes, so Algorithm 1 runs in O(n).

Output. Clean strokes, each with 32 points, and normalized images.

3.7 Feature Extraction

Features describe how a sample differs from the reference of its target character. Table 13 lists them.

Table 13: Features Used by the Validators

Version	Feature	Definition	Detects
1	Form similarity	Dice similarity between the sample image and the reference image	Overall form
2	Stroke count difference	Number of drawn strokes minus number of reference strokes	Count
2	Order match	Whether the strokes are paired with the reference strokes in the same sequence	Order
2	Reversed strokes	Strokes whose reversed copy is closer to the reference than the original	Direction
2	Shape distance	Distance between each paired stroke and its reference stroke	Form (stroke shape)
2	Kudlit offset	Difference between the kudlit position and the reference kudlit position	Kudlit placement

Form similarity. For two binary images A and B, the Dice similarity [32] is

$$Dice(A, B) = 2 \times |A \cap B| / (|A| + |B|)$$

where |A| and |B| are the numbers of ink pixels and |A ∩ B| is the number of pixels that are ink in both images. Both images are dilated by 1 pixel (with a 3 × 3 block) before the comparison, so that small shifts are tolerated.

Shape distance. For a drawn stroke $s = (p_1, \ldots, p_N)$ and a reference stroke $r = (q_1, \ldots, q_N)$,

$$d(s, r) = (1 / N) \times \Sigma \, \|p_i - q_i\|, \text{ for } i = 1 \text{ to } N$$

where $\|p_i - q_i\|$ is the straight-line distance between the matching points. The distance is in units of the character size.

Direction. The reversed copy of s is rev(s) = (p_N, ..., p_1). A stroke is drawn in the wrong direction when d(rev(s), r) < d(s, r).

Pairing and order. Strokes may be drawn in the wrong order, so each drawn stroke is first paired with a reference stroke by shape. The distance used for pairing is min(d(s, r), d(rev(s), r)), so a direction error does not disturb the pairing. All possible pairings are tried, and the pairing with the smallest total distance is kept. The order is correct when the kept pairing matches the drawn strokes to the reference strokes in the same sequence.

Kudlit offset. For a kudlit with center (x_k, y_k), and a main body with bounding-box center (x_m, y_m), width w_m, and height h_m, the relative position is

$$c = ((x_k - x_m) / w_m, (y_k - y_m) / h_m)$$

and the kudlit offset is $\delta = \|c_{sample} - c_{reference}\|$. If w_m or h_m is 0, it is replaced by 1. This follows the idea of comparing spatial relations between strokes in [19].

Input. Normalized images, clean strokes, and reference strokes. **Output.** A set of feature values for each sample.

3.8 Model Implementation

3.8.1 Data Split and Threshold Setting

Input. The expert-reviewed dataset and its feature values.

Activities.

1. Split the writers, not the samples, into a development set (80%, which is 16 writers) and a test set (20%, which is 4 writers). The writers are drawn at random with a recorded seed, and no writer is in both sets.
2. Inside the development set, use 4-fold cross-validation by writer (4 writers per fold).
3. Set three thresholds from data: τ_{form} (Version 1), τ_{shape}, and τ_{kudlit} (Version 2). For each threshold, the candidate values are 50 equally spaced values between the smallest and largest value of its feature in the development set. For each candidate, compute the F1-score on each validation fold, and average the four scores. Keep the candidate with the highest mean F1 (the smaller value if there is a tie). A sample is flagged when its distance or offset is above the threshold, or when its form similarity is below τ_{form}.
4. For τ_{shape} and τ_{kudlit}, the positive samples are those whose expert labels include a form error or a kudlit error, and the negative samples are those with no error. For τ_{form}, the positive samples are all invalid samples and the negative samples are all valid ones, so that the baseline is tuned on the same overall task.

5. Fix the three thresholds. The test set is not used until Section 3.9.

Output. Three fixed thresholds. The count, order, and direction checks are exact and have no threshold.

3.8.2 Version 1: Form Validator

Input. The normalized image of the drawing and the target character.

Activities. Get the reference image of the target character, dilate both images by 1 pixel, compute the Dice similarity, and return form-valid when it is at least τ_{form}, and form-invalid otherwise. Version 1 sees only the final image, so it cannot tell how the character was drawn.

Big O. For an image of P pixels (64 × 64), the comparison costs O(P), which is constant for a fixed image size.

Output. A form-valid or form-invalid result.

3.8.3 Version 2: Stroke Validator

Input. Normalized strokes, the reference strokes of the target character, and the thresholds τ_{shape} and τ_{kudlit}.

Activities. Algorithm 2 runs the five checks. If the stroke counts differ, the count error is reported and the other checks are skipped. The order is not scored for characters marked in Section 3.5.1.

```
Algorithm 2: Stroke checks (Version 2)
Input : drawn strokes S' = (s_1, …, s_m), reference strokes R = (r_1, …, r_K),
        kudlit index κ (or none), thresholds τ_shape and τ_kudlit
Output: list E of failed checks

1:  E ← empty list
2:  if m ≠ K then
3:      add "count" to E; return E
4:  end if
5:  for i ← 1 to m and j ← 1 to K do
6:      D[i][j] ← min(d(s_i, r_j), d(rev(s_i), r_j))
7:  end for
8:  π ← the pairing of drawn strokes to reference strokes with the smallest
        sum of D[π(j)][j]
9:  if π is not the same-sequence pairing then add "order" to E
10: for j ← 1 to K do
11:     s ← s_π(j)
12:     if d(rev(s), r_j) < d(s, r_j) then add "direction (stroke j)" to E
13:     if D[π(j)][j] > τ_shape then add "form (stroke j)" to E
14: end for
15: if κ exists and kudlit_offset(S', R, π, κ) > τ_kudlit then add "kudlit" to E
16: return E
```

Big O. Let K be the number of reference strokes and N the number of points per stroke. Steps 5 to 7 compute K × K stroke distances at O(N) each, which is $O(K^2N)$. Step 8 tries K! pairings at O(K) each, which is O(K · K!). Steps 10 to 15 cost O(KN). Together with Algorithm 1, the total is $O(n + K^2N + K \cdot K!)$. K is small and fixed by the character specification, so the cost in practice grows with the number of points n.

Output. A list of failed checks, empty when the sample is stroke-valid.

3.8.4 Hybrid Decision and Evidence Retrieval

Input. A drawing, its final image, and the selected target character.

Activities. The final result is valid only when Version 1 passes and Version 2 reports no failed check. Otherwise the result is invalid, and the failed checks are the detected errors. The retrieval layer then runs after the result is known. For each failed check, it looks up the character record, the matching stroke or kudlit rule, the feedback template of the error type, and the linked source. Retrieval-augmented generation combines a language model's own memory with retrieval from an external source of knowledge [33]. Here the knowledge graph is the external source. The explanation is produced by a fixed template for each error type, filled with the retrieved facts, for example "Stroke {n} of {character} should be drawn {direction description} [{source}]." A language model may be used only to reword the filled template into a short sentence. It receives the retrieved text and the result, and it cannot change either. Figure 3 shows the flow, and Algorithm 3 gives the steps.

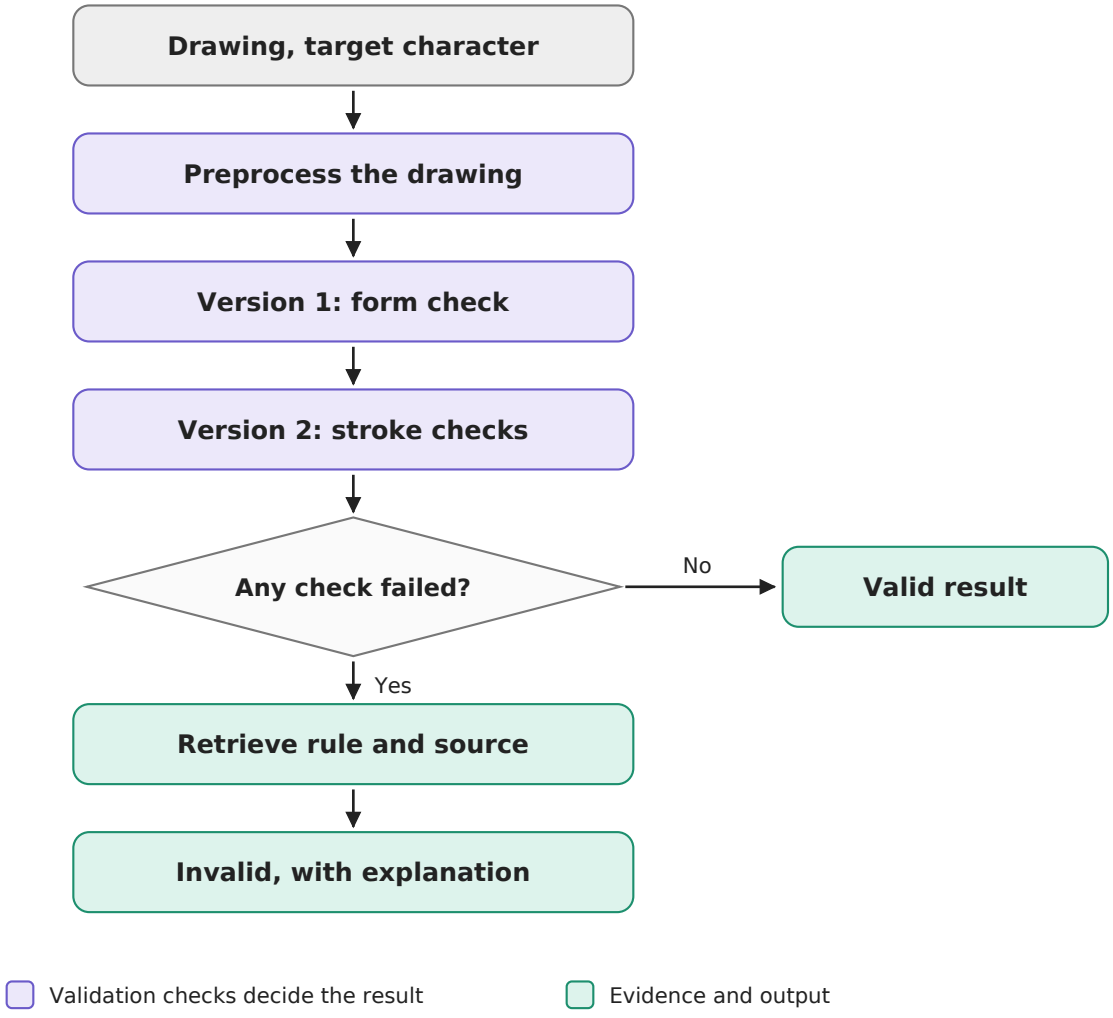


Figure 3: Hybrid Validation Flow

```
Algorithm 3: Hybrid validation
Input : raw strokes S, canvas image I, target character c
Output: result (valid or invalid), failed checks E, explanation

1:  S' ← Preprocess(S); I' ← normalize_image(I)
2:  (R, refImage, κ) ← KnowledgeGraph.reference(c)
3:  E ← empty list
4:  if Dice(dilate(I'), dilate(refImage)) < τ_form then add "form" to E
5:  E ← E + StrokeChecks(S', R, κ, τ_shape, τ_kudlit)
6:  if E is empty then return "valid", E, "No deviation found"
7:  evidence ← empty list
8:  for each failed check e in E do
9:      add KnowledgeGraph.retrieve(c, e) to evidence
10: end for
11: explanation ← fill_template(evidence)
12: return "invalid", E, explanation
```

Big O. The cost is $O(n + P + K^2N + K \cdot K! + |E| \cdot q)$, where P is the number of image pixels, $|E|$ is the number of failed checks, and q is the cost of one knowledge-graph lookup. Lookups are made by character ID and error type, so q does not grow with the number of samples.

Output. A valid or invalid result, the failed checks, and an explanation with its source.

3.8.5 Summary of Fixed Parameters

Table 14: Summary of Fixed Parameters

Parameter	Value
Canvas size and pen width	400 × 400 pixels, 8 pixels
Volunteer writers (W)	20
Attempts per character	4 (2 correct, 2 controlled-error)
Smoothing window	3 points
Points per stroke (N)	32
Normalized image	64 × 64 pixels, longest side 56 pixels
Dilation before image comparison	1 pixel
Split by writer	80% development (16 writers), 20% test (4 writers)
Cross-validation	4 folds by writer, inside the development set
Threshold search	50 equally spaced values; highest mean validation F1-score
Thresholds set from data	τ_{form} (Version 1), τ_{shape}, τ_{kudlit} (Version 2)
Random seeds	Fixed and recorded

3.9 Performance Evaluation

This section describes how the validators and the evidence retrieval will be assessed on the test set. The test set is used once, after the thresholds are fixed.

3.9.1 Ground Truth and Evaluation Design

Every decision is compared with the expert-reviewed label. The invalid class (a formation error is present) is the positive class, because the validator's main job is to catch errors. The same test samples are used to evaluate Version 1 alone, Version 2 alone, and the complete hybrid system. Results are also reported separately for controlled-error samples and for natural errors (Section 3.5.4).

3.9.2 Metrics

The metrics are computed from the counts of true positives (TP, an invalid sample flagged invalid), true negatives (TN, a valid sample accepted), false positives (FP, a valid sample flagged invalid), and false negatives (FN, an invalid sample accepted as valid).

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

A confusion matrix is reported for each validator.

3.9.3 Error-Specific Results

For each error category in Table 11, two values are reported:

Detection recall = (samples with the error that were flagged invalid) / (samples with the error)

Identification accuracy = (samples with the error whose reported errors include that category) / (samples with the error)

Comparing the two validators by category shows where the stroke checks add something that the final image cannot show.

3.9.4 Evidence Retrieval

A random sample of 100 invalid outputs from the test set (or all of them if there are fewer) is given to the expert. For each output, the expert answers whether the retrieved rule and source are the correct ones for the character and the error (source correctness, yes or no), and rates how well the explanation describes the actual error and is clear to read (feedback relevance, 1 to 5). The share of correct sources, the mean rating, and the share of ratings of 4 or 5 are reported.

3.9.5 Response Time

Response time is the time from the moment a drawing is submitted to the moment the result with its explanation is returned. It is measured in the back end for every test sample on the same computer, and the mean and the maximum are reported. The computer's specifications are recorded.

3.9.6 Statistical Treatment

Results are summarized as counts and percentages. Version 1 and Version 2 are compared on the same test samples, using whether each decision matched the expert label, with McNemar's test [34] at a 0.05 level of significance. Because the test set has few writers, the number of samples behind every result is reported.

3.10 System Development Method and Model Utilization

To show that the validators work in use, they are built into a web prototype. The user selects a target character, draws it on the canvas, and receives the result and an explanation with the reference image and its source (Figure 3). The front end uses React with the HTML5 Canvas, and the back end uses FastAPI, where preprocessing, both validators, and the retrieval run. The prototype is a validation tool only and not a learning platform, as stated in the scope of the study. Version 1 establishes form conformance, and Version 2 delivers the stroke-validation claim of the thesis.

The system is built with the Agile iterative method [35]. The method fits the study because the results of each iteration, such as the first labeled samples, change what is collected and tuned next. At the end of every iteration, the researchers and the expert review the output, record the changes, and decide what to correct in the next one. Table 15 shows the six iterations.

Table 15: Agile Iterations

Iteration	Activities	Input	Output	Obj.
1. Requirements and standards	Select the supported characters, convert the KWF material into character specifications, define valid and invalid writing, and prepare the expert rubric	KWF material, expert	Approved character specification and rubric	1
2. Knowledge base	Build the knowledge graph with character records, reference images, writing and kudlit rules, common errors, and sources	Specification, reference images	Queryable, source-traceable knowledge base	1
3. Version 1 form validator	Build the drawing canvas, store the final image and the raw event data, normalize the drawing, and compare it with the reference	Canvas, reference images	Working form-conformance prototype	2
4. Dataset collection and labels	Collect correct and controlled-error samples, capture coordinates, timestamps, and pen events, and have the expert label them	Task sheets, writers, expert	Expert-reviewed stroke-level dataset	1

Iteration	Activities	Input	Output	Obj.
5. Version 2 stroke validator	Segment strokes, extract features, and compare order, direction, trajectory, spatial relations, and kudlit placement with the reference	Dataset, reference strokes	Hybrid stroke-based validator	2
6. Evaluation and integration	Test on held-out writers, compute the metrics, review the errors, and connect the feedback to the knowledge graph and retrieval	Test set, validators	Final prototype and results for Chapter 4	3

3.11 Ethical Considerations and Data Management

The study will follow the research ethics guidelines of Mariano Marcos State University and will seek ethical clearance before data collection. It will also follow the Data Privacy Act of 2012 [36].

Informed consent. Each volunteer receives an information sheet explaining the purpose of the study, what is recorded, and that taking part is voluntary and can be stopped at any time. Each volunteer signs a consent form before writing.

Anonymization. Samples are stored under writer codes. Names appear only on the consent forms, and the list linking names to codes is kept separately by the lead researcher.

Restricted access. The data are kept in a password-protected location that only the researchers and the expert can open. The reference materials of KWF and the expert are used with their permission, are cited, and are not shared outside the group.

Traceability. Every sample record keeps the writer code, session, device, character, attempt number, the labeler, and the label date. Each evaluation run records the version of the character specification, the threshold values, the random seeds, and the software version. Every knowledge graph record is linked to its source. Changes are logged with their dates.

The validator is an aid to checking and not a substitute for the judgment of Baybayin experts.

References (continuing from Chapter 2, ACM format)

[30] [Author]. [Year]. *[Title of the expert's writing guide]*. [Publisher], [Place]. (Fill in from the book itself.)

[31] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. Knowledge Graphs. *ACM Computing Surveys* 54, 4, Article 71 (June 2021), 37 pages. https://doi.org/10.1145/3447772

[32] Lee R. Dice. 1945. Measures of the Amount of Ecologic Association Between Species. *Ecology* 26, 3 (1945), 297–302.

[33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Curran Associates, Inc., 9459–9474.

https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[34] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[35] Kent Beck et al. 2001. Manifesto for Agile Software Development. Retrieved [date accessed] from https://agilemanifesto.org/

[36] Republic of the Philippines. 2012. Data Privacy Act of 2012, Republic Act No. 10173. Retrieved [date accessed] from https://www.officialgazette.gov.ph/2012/08/15/republic-act-no-10173/

Citations [2], [7], [8], [13], [17], [18], and [19] refer to the Chapter 2 reference list.